

Chapter 13

Interactive Cellular Automata Systems for Creative Projects

Angus Graeme Forbes

Abstract. The use of cellular automata as a mechanism for generating captivating and unpredictable creative output has been explored by artists and composers since the introduction of the “Game of Life” by the mathematician John Conway and its subsequent popularization via the writings of Martin Gardner. This chapter surveys notable creative projects that incorporate cellular automata systems in order to generate new aesthetic possibilities. In particular, it describes a recent project, *Fluid Automata*, that utilizes a cellular automata system to create fluid simulations and to enable a range of expressivity in different interactive environments.

13.1 Creative Projects Based on Cellular Automata Systems

Cellular automata (hereafter, CA) systems have been used as a mechanism for generating creative output by artists and researchers since the introduction of the “Game of Life” by the mathematician John Conway and its subsequent popularization via the writings of Martin Gardner [33]. These include practitioners in a wide range of fields, such as musical composition [7, 15, 20], urban design [58], cognitive science [66], video game programming [3, 5], and media arts [12, 24, 41].

A rich tradition of using algorithms to generate creative content exists in both music composition and the visual arts. For instance, artists such as Duchamp and writers and artists of the Oulipo movement placed constraints on their art-making process in order to generate more focused possibilities from a reduced set of choices [17, 43]. The artist Sol LeWitt created algorithmically inspired instructions for the creation of his paintings [39]. Composers from Mozart to John Cage have used randomness as a generator of musical themes [49]. More recently, 20th-century composers, including Xenakis, have developed complex algorithmic compositional systems that guided the creation of their musical scores [67].

Angus Graeme Forbes

University of Arizona, USA

e-mail: angus.forbes@gmail.com

<http://angusforbes.com>, <http://creativecodinglab.com>

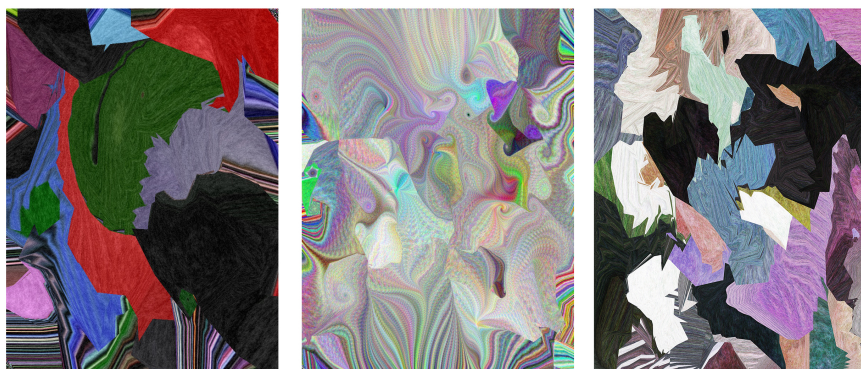


Fig. 13.1 Examples of output from an iPad application utilizing the Fluid Automata system showing the wide range of aesthetic possibilities generated by the system when using different fluid profiles defined by customizing the fluid, visualization, and noise parameters

In a series of essays about the rise of new media art in the 1960s, Edward Shanken explores the ways in which the first media artists developed art installations that were influenced by “cybernetic systems” [52]. These cybernetic works encouraged viewers to think about the relationship between hardware and software, and the metaphorical parallels between the physicality of structure and the emergence of consciousness [53]. Conway’s “The Game of Life” explicitly invokes the idea that emergent, like-like structures emerge over time out of simple rules, similar to the way that human consciousness, through a much more complicated series of steps, evolved over of millennia of mutation and selection.

An attractive aspect of CA is its simplicity. Through their discretization, time and space are simplified; everything happens simultaneously, in uniform steps, and within a small uniform grid. Thus the complexity that emerges from the simple rule set governing the system is, in a sense, more tractable. An array of unexpected structures with seeming agency and life-like behaviors (e.g., breeders, gliders, rakes, replicators, etc.) can be explored, analyzed, and modified. For instance, the researcher Dave Greene recently indicated that he successfully built a CA system (in a grid of $14,826,990^2$ cells) that creates a complete copy of itself in 237,228,617 iterations [34].

Some artists have directly incorporated CA systems into their work. These include Dale Millen’s *Cellular Automata Music* [42] and a series of compositions by Peter Beyls that explore the mapping of CA systems to musical structure [8]. Iannis Xenakis used CA in some of his later compositions [54, 67]. Surveys by Dave Buraston and Ernest Edmonds provide a more thorough review of the history of CA systems and composition [13, 14]. Philip Galanter, Margaret Boden, and Ernest Edmonds discuss how CA is more generally situated in the context of “generative art,” demonstrating that other processes with emergent behavior are similarly inspirational to artists, including: Belousov-Zhabotinsky reactions [19]; fractal geometry; chaotic attractors; Lindenmayer systems [45]; flocking algorithms; and many other

self-organizing systems based on iterative processes [10, 28]. In particular, Philip Galanter defines a clear distinction between rule-based art and generative art. Rule-based art may not have the “sufficient specificity or autonomy to allow the artist to give up control to the rules, and allow [the rules] to take over and produce the art” [28]. That is, he writes, “the key element in generative art is then the system to which the artist cedes partial or total subsequent control” [27].

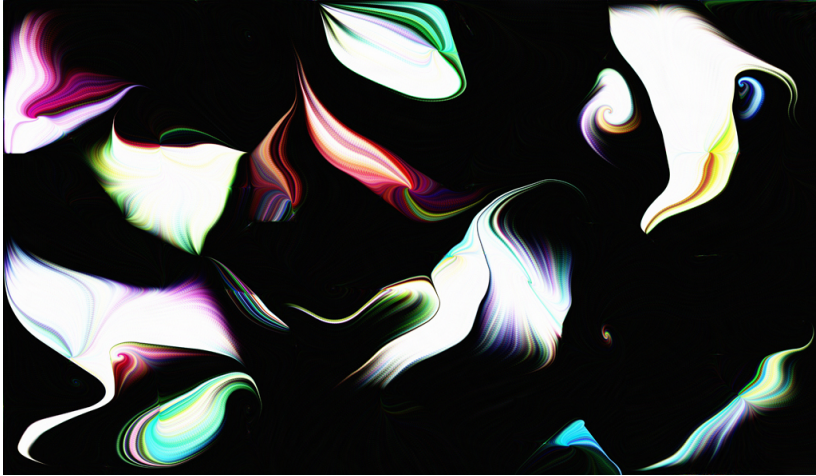


Fig. 13.2 Example output using a custom color palette with high-contrast image processing variables

Various artists and researchers have extended the type and number of elements in CA systems beyond the basic 2D, binary system introduced by Conway. For example, Ritesh Lala’s artwork, *Morphon*, uses a 3D CA system to create pleasing visual effects [40]. An implementation developed by Stephan Rafler called “SmoothLife” implements “circularly shaped integrals” in order to situate CA in the continuous domain [46]. Other modifications use non-uniform grids or multiple colors in order to achieve particular effects, such as cyclical behaviors [38]. Paul Brown uses CA systems to develop his “kinetic paintings” [1], and numerous recent contemporary works use or represent generative systems as a primary component. For instance, Casey Reas has created a series of generative works collected in his “Process Compendium Catalog” [47] and described in his “Process Lecture” [48]. George Legrady has produced works that represent generative processes, such as *Kinetic Flow* [41], and one installation, *Blink*, demonstrates “the process of self-organization in a vast matrix of eyes that open and close according to their neighbors’ behaviors” [44]. Philip Galanter has also produced generative artworks, including his lightbox drawings and interactive sculptures [29, 30].

Interactive simulations of CA have been available as software since the 1970s, and some artworks using CA methods also incorporate direct interaction. In most cases, the interaction is straightforward, allowing a user with a mouse to change

the state of single cell, or perhaps to add a “gun” or “glider” to the CA system at a particular location. A somewhat more involved way to interact with a CA system is demonstrated in a series of works by the installation artist Bill Vorn. Vorn presents a large CA grid that can be controlled by standing at different locations throughout the installation which are mapped to the cells in the CA system [61]. The artist Charlie Roberts also uses a CA grid as an example of a dynamic interface used to control networked compositions via mobile devices [50, 51].

The next section introduces the Fluid Automata system, based on CA, and subsequently describes technical details about the system, including the simulation engine, the visualization methods, and the single-user and multi-user interaction techniques used in different deployments. It then describes a collaborative art installation, *Annular Genealogy*, involving musical and visual outputs that is based on the Fluid Automata system.

13.2 The *Fluid Automata* Project

The *Fluid Automata* system explores the use of fluid simulation as an aesthetic component for interactive art. While fluid simulation is often used in scientific visualization applications in order to provide a practical understanding of how fluids behave in particular contexts, the use of fluid simulations to create realism and excitement in entertainment contexts is also a widely explored topic. Moreover, fluid systems have been used as a technique in generative media arts contexts, and a number of recent art projects utilize fluid simulation as a component of the work. A method created by Jos Stam in 1999 to create a stable fluid system first made it possible to represent realistic looking fluids at real-time frame rates [55, 56]. Many interactive artworks have made use of this particular technique. For instance, Memo Atken has created a series of demonstrations based upon Stam’s method, showcasing them using mobile devices for interaction and making the code available for the OpenFrameworks and Processing multimedia frameworks [4]. Another project that incorporates Stam’s method is Graham Wakefield and Haru Ji’s *Artificial Nature*. This project uses computer vision techniques to allow participants to interact with a 3D fluid representation through the movement of their bodies [62]. The *Fluid Wall* installation by Naureen Mahmood and Austin Hines similarly enables users to manipulate a wall of fluids using a Kinect interface [6]. Other fluid simulation methods, such as one presented by Martin Guay, et al., are optimized for real-time interaction in video games [36]. Although simulation methods generally focus on producing accurate representations of natural systems, the Fluid Automata system demonstrates that aesthetically-interesting visuals with a wide variation of movement and color can be produced from a simple set of rules that do *not* attempt to exactly reproduce natural systems. In this sense, although influenced by physical simulation, Fluid Automata could more accurately be considered a work of generative art that utilizes a custom CA system [10, 27].

Scientific visualization projects aim to help researchers identify and reason about salient aspects of their data. While an aesthetic sensibility may contribute to the

success of a visualization technique, this is not normally the primary motivation for its creation. Media artists, on the other hand, generally place aesthetic considerations at the forefront of their concerns. Similarly, physical simulations are concerned with accuracy and realism rather than extensibility and interaction, which are central to media arts. However, media arts projects, due to time constraints or limitations in technical knowledge, often incorporate readily-available techniques not originally intended for artistic production, and therefore not as easily adaptable to artistic situations. Fluid Automata provides a flexible system that can be used in many situations, including on less powerful mobile devices, while retaining enough realism to be useful for simulations. Figure 13.1 shows examples of different outputs created with the Fluid Automata system.

13.3 Simulation and Visualization in the *Fluid Automata* Project

The Fluid Automata system is composed of three integrated components: a fluid simulation engine; a flow visualization technique; and an interface that encourages interaction with the visualized fluid simulation [25]. Here I look at the technical details of the fluid simulation and flow visualization components that are shared across multiple deployments of the project.

13.3.1 *Fluid Simulation*

Fluid Automata introduces a novel fluid simulation technique, inspired by CA systems. While its title was inspired by Tibor Gánti's discussions of "chemotons" [31, 32], the initial kernel of insight for the Fluid Automata system arose from thinking about how to create a simple rule-based system to produce emergent behavior. CA systems demonstrate complex behavior emerging through an iterative system that updates the state of each element (positioned in a uniform grid) based on a set of basic rules. These rules determine the next state of each element by querying each of its neighbors. In Conway's original automata system, each pixel has a binary state, and either "lives" (is set to 1) or "dies" (is set to 0) based upon the number of surrounding pixels that are on or off. Other CA systems, including many that are examined in Stephen Wolfram's "A New Kind of Science" [64], utilize different rule sets and involve multiple states. In particular, a range of CA approaches to fluid simulation have been introduced. For instance, an early paper by Gerard Vichniac describes discretized models for fluid simulation based on CA, but indicates that precise values for specific parameters are necessary in order to avoid "divergences" [60]. Another paper surveys different CA paradigms for fluid simulation, comparing lattice gas, digital fluid, and lattice-Boltzmann strategies [16]. A 1998 paper describes 3D fluids that can be represented with hexagonal and rhombododecahedron grids instead of uniform squares or cubes [11]. Stephen Wolfram also explored general non-linear approximations of fluids via CA [63].

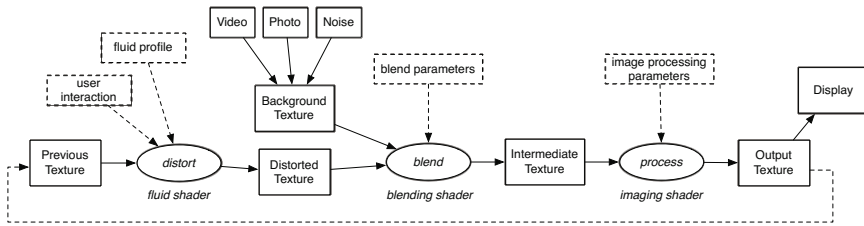


Fig. 13.3 Schematic for the main functionality of the Fluid Automata system. The output texture after one timestep becomes the input for the next timestep.

Unlike Conway's Game of Life and other early CA in which each cell contained only binary information, Fluid Automata's 8-bit CA system contains $2^8 \cdot 2^8$ states per cell: 256 values for the magnitude of a fluid vector and 256 states used to describe its orientation. Retaining the discretization and simplicity central to CA systems makes it possible to create a complex system that is linear and replicable and that effectively simulates the movement of fluid. Unlike the majority of commonly-implemented fluid simulations which utilize the non-linear Navier-Stokes equations, the Fluid Automata algorithm is always stable at any length of timestep. That is, the system is inherently non-realistic since it lacks a mass conservation condition and because the fluid is compressible. Although the system is not physically accurate, it has the advantages of being easy to modify in real-time and relatively straightforward to implement, leading to its incorporation in a range of projects. It distributes a flow of energy throughout the system as follows:

1. The screen is divided into a grid of cells.
2. New energy is added into the grid by user interaction.
3. The energy at each cell is split into three streams, a forward stream and a left and a right stream.
4. In each of the defined directions for each stream defined in step 3, the energy of each cell is moved into the neighboring cell via the following process:
 - a. The cell is displaced along the vector describing the energy stream.
 - b. For each neighboring cell the displaced cell intersects with, create a "partial" vector by scaling the original vector with the amount of intersection.
 - c. This partial is added to the cell it intersects with.
5. When this process has been completed for the entire grid, the partials associated with each cell are combined, creating a new vector that replaces the current vector in the cell.
6. Energy is removed from the system by scaling the energy in each cell by a dampening factor.
7. Steps 2 through 6 are iterated at each timestep until there is no energy left in the system.

More formally, the fluid system acts on a grid G of cells $C_{i,j}$ each containing an energy vector $\vec{E}_{i,j}$, with a particular magnitude m and orientation θ , and where $0 < i < \text{columns}$ and $0 < j < \text{rows}$. In order to define a “fluid profile” for the system, two ratios are introduced that regulate that behavior of energy as it moves through the cells. The first, the *momentum* ratio, or r_1 , defines how much energy moves forward versus moving to the sides. The second, the *directionality* ratio, or r_2 , defines how much energy moves to the left versus moving to the right. A parameter, *angularity*, is defined as $\angle\phi \in [0, \pi)$, describing a rotation offset from $\angle\theta$. Additional parameters influencing the fluid profile are the *viscosity* of the system, which acts as a dampening factor defining the rate at which energy is removed from the system, and the *sensitivity* which controls how much energy is added to the system through some form of user interaction. New energy is added into the cells in a particular direction using the multi-touch capabilities of the tablet device. The amount of energy that is added in a particular touch depends upon the *sensitivity* parameter, which can be adjusted during runtime. The magnitude of energy is also determined by how far the current position of the touch is from its previous position. This difference also determines the direction of the added energy. If there is no change in position, then the energy is added in the last known direction. This vector of new energy is added with any existing energy at the currently touched cell to update $\vec{E}_{i,j}$.

At each timestep t during the operation of the fluid system, the energy $\vec{E}$ in $C_{i,j}$ is split into three separate streams: a forward stream, $\vec{F}$, and two “orthogonal” streams, $\vec{L}$ and $\vec{R}$. Using the current fluid profile (the values for the parameters of *momentum*, *directionality*, and *angularity*), and the current magnitude and orientation for each cell, I define these three streams like so (in Equations 3.1 through 3.3):

$$\vec{F} = \begin{pmatrix} r_1 m \\ \theta \end{pmatrix} \quad (13.1)$$

$$\vec{L} = \begin{pmatrix} (1 - r_2)(1 - r_1)m \\ \theta + \phi \end{pmatrix} \quad (13.2)$$

$$\vec{R} = \begin{pmatrix} r_2(1 - r_1)m \\ \theta - \phi \end{pmatrix} \quad (13.3)$$

Figure 13.5 shows examples of how the current total energy in a cell is split into three streams based on these parameters. Every cell in G thus contains three separate streams of energy, $\vec{F}$, $\vec{L}$, and $\vec{R}$. These streams are used to define the flux of energy moving from each cell into its neighboring cells. The calls to the `STREAMVECTOR` procedure in the code listing in the Appendix also describe how energies are split into different vectors based on r_1 and ϕ .

A “displaced” cell $D(C, \vec{v})$ is a copy of a cell $C \in G$ that moves along a vector $\vec{v}$ positioned at the center of C . This displaced cell D intersects with between 1 and 4 cells (the original cell C itself and up to three neighboring cells). The magnitude of any energy vector is constrained to range between 0 and 1. When displacing a cell, the magnitude is scaled by the length of a cell side. For instance, if the grid is

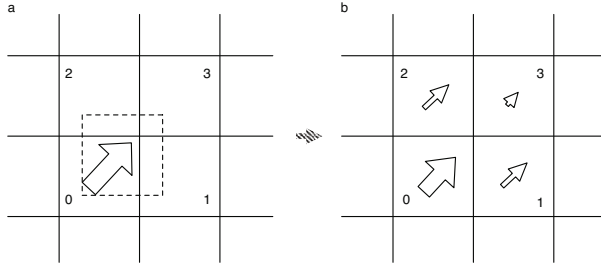


Fig. 13.4 Example showing how a single stream of energy in a single cell is distributed to other cells. In this example, the left grid (a) indicates that the energy in cell 0 is pointing in the direction $\pi/4$ with a magnitude of 0.5. The dotted box shows where the “displaced” cell intersects with its neighbors. The largest intersection of energy stays within the original cell (cell 0); a tiny amount of energy is pushed into cell 3; and a small amount of energy is pushed into cells 1 and 2. The right grid (b) shows the distribution of energy from this single cell moved into its neighbor cells and back into itself.

divided into 10x10 cells, then each cell side is .1 in length, and so the magnitude in a cell is scaled by 1/10th before being displaced. This ensures that a displaced cell will only ever intersect a cell that is its immediate neighbor (although exceptions to this constraint may have interesting creative possibilities.) The amount of overlap between the displaced cell and the cell it intersects with determines how energy is placed into that cell. An intersection $I(D, C)$ is simply the amount of overlap between the displaced cell and another cell (Equation 4). The value of any intersection can range between 0.0 (no intersection) and 1.0 (full overlap).

$$I(D, C) = \left(\text{Area}(D) \cap \text{Area}(C) \right) / \text{Area}(C) \quad (13.4)$$

This value is used to create a “partial” vector $\vec{p}$ via a function $P(N, \vec{v}, C)$ for each energy stream. This partial is calculated simply by scaling the stream by the amount it overlaps with the current cell, as defined in Equation 5:

$$P(N, \vec{v}, C) = I(D(N, \vec{v}), C) * \vec{v} \quad (13.5)$$

In Equation 5, N refers to a neighbor cell of a cell $C \in G$ and $\vec{v}$ refers to one of the energy vectors ($\vec{F}$, $\vec{L}$, or $\vec{R}$) in that neighbor cell. Figure 13.4 depicts an example of this displacement and the subsequent generation of partials. For each cell $C_{i,j}$ in G , each neighbor is examined to determine how much each of its three streams overlap. The partials created by any intersection between the neighbor streams are summed to generate the new energy vector for the cell (Equations 6 through 9). It is important to note that a cell is considered to be a neighbor of itself. For instance, if a vector of magnitude 0.5 is pushed upwards at 90 degrees, it would intersect with both the current cell and the neighbor cell above it. Since the displaced cell would move a distance of 50% of its height from its current position, it would end up intersecting the current cell and the neighbor cell equally, and thus a copy of the vector, scaled by

50%, would be placed in each of the cells. Figure 13.4 shows a typical example of a single stream of energy in a single cell being distributed to its neighbors. Excluding any dampening factor or any new energy added from user interaction, the system will retain exactly the same amount of energy over each timestep. The PARTIAL and MAKERECT procedures in the code listing provided in the Appendix to this chapter also describe how partials are created through this displacement process.

$$\vec{F}_{i,j,t} = \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} P(C_{p,q}, \vec{F}_{p,q,t-1}, C_{i,j}) \quad (13.6)$$

$$\vec{L}_{i,j,t} = \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} P(C_{p,q}, \vec{L}_{p,q,t-1}, C_{i,j}) \quad (13.7)$$

$$\vec{R}_{i,j,t} = \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} P(C_{p,q}, \vec{R}_{p,q,t-1}, C_{i,j}) \quad (13.8)$$

$$\vec{E}_{i,j,t} = \vec{F}_{i,j,t} + \vec{L}_{i,j,t} + \vec{R}_{i,j,t} \quad (13.9)$$

Other parameters can also be adjusted to create different characteristics for a particular fluid profile. These include: controlling the “jitter”, or randomness of the system, and clamping the maximum outflow for the cells within the grid. I also experimented with a toroidal representation of the system where fluid energy wraps around the edges of the screen, instead of bouncing off the edges (that is, $x_{max+1} = x_0$ and $x_{min-1} = x_{max}$). Setting a maximum outflow parameter interestingly creates the sense of ice cracking and melting when a particular threshold is exceeded. Different settings of *viscosity* and *angularity* can create more or less turbulent behaviors. The iterative nature of the system does in fact create a wide variety of fluid-like structures, including the creation of eddies, vortices, and turbulence. Figure 13.1 and Figure 13.6 show examples of fluid systems with different fluid profiles defined by different settings for the *angularity* and *directionality* parameters. The Appendix to this chapter provides a code listing that describes how a cell is updated via querying its neighbors.

Just as simulations for realistic films and video games do not feel constrained by a perfect representation of the physics of a visual effect, so to should media artists not feel constrained by a perfect representation of existing algorithms and equations for a particular kind of effect. By creating a custom fluid system with a wide range of parameter adjustments I was able to extend the aesthetic applicability and variation of the fluid system to capture unusual behaviors not normally depicted with fluid representations. Although the system appears realistic, it in fact sacrifices physical accuracy in order to emphasize interactivity, expressiveness, and experimentation.

13.3.2 Flow Visualization

The main image processing scheme in the Fluid Automata system is based on a feedback loop whereby a high-resolution background image is perpetually blended

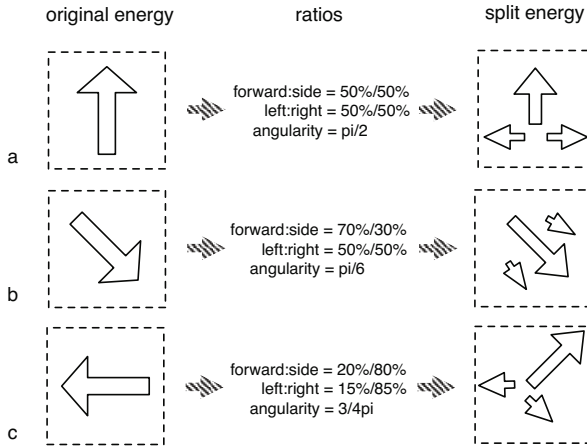


Fig. 13.5 Examples of fluid systems with different characteristics, i.e., fluid profiles, based on different settings. The settings can be changed in real-time. The left side of the chart represents the energy in a single cell; the right side of the chart shows how the energy is split into different streams based upon the parameters defining the ratio between forward and orthogonal momentum; the ratio between left and right momentum; and the angularity of the orthogonal momentum. In the top row (a), energy with a magnitude of 255 and an orientation of $\pi/2$ is split evenly between forward and orthogonal momentum. In the middle row (b), most of the energy in the cell remains moving in the original direction; the orthogonal energy is close to the forward orientation. In the bottom row (c), most of the energy is moving to the side, with a high angularity, and with an uneven distribution between the left and right sides. (Note, the arrows representing energy vectors are not drawn exactly to scale.)

together with a distorted version of itself. The characteristics of the distortion are based directly on the current state of the fluid system, which cause pixels to smear or streak in the direction of the flow vectors. This is similar to Jarke J. van Wijk's "Image Based Flow Visualization," which has been extended for use in a variety of scientific visualization applications, including animated and 3D flows [57, 59].

Since a focus of Fluid Automata is aesthetic exploration, implementations provide the user with a variety of tools to alter aspects of these blending operations, and in addition introduce an image processing layer whereby the user can change a variety of parameters, including: the rate and amount of blending, the type and quality of the background texture, and the brightness, contrast, and saturation of the blended image. The default background texture is a grayscale noise texture at a resolution exactly matching the display, but various other options are possible, including: lower resolution textures; static colored textures; static image textures; dynamic textures that are updated by noise functions; and dynamic textures that are updated by a live video feed. Figure 13.2 shows an image created using a static colored background texture with high contrast and high saturation image processing



Fig. 13.6 Details of high-resolution output demonstrating “unrealistic” fluid simulation using the *Fluid Automata* system. The left image shows the addition of a high amount of vorticity; the right image shows the “spikiness” associated with a high amount of energy.



Fig. 13.7 Photo of iPad application using the Fluid Automata system with a live video feed replacing the background noise texture

parameters. Figure 13.7 shows an image that is created using a live video feed as the background image, rather than an image populated with randomly-colored pixels.

Figure 13.3 provides an overview of the fluid visualization process over the course of a single frame. At t_i , the system: a) distorts the previous image texture (if $i > 0$, otherwise it uses a copy of the background texture) based on user interaction and the current fluid profile; b) blends the background texture in with the distorted texture based on a blending parameter (that can be updated in real-time by a user); and then c) applies image processing filters (based on the image processing parameters described above) to the image to create a final output texture for this frame at t_{i+1} . This output texture is then used as the input texture for the next frame.

13.4 Single-User Interaction Techniques

The Fluid Automata system enables a wide range of interaction techniques. It was originally implemented in a stand-alone version that could be controlled by a single user on an iPhone or iPad [23]. The primary interaction component utilizes the multi-touch capabilities of iOS devices, creating a believable representation of fluids being disrupted by human touch. Users are also able to manipulate a wide number of parameters to control the fluid profile of the simulation as well as a number of image processing components. Much experimentation went into making the user interaction with the fluid system feel responsive and inviting: by tapping the screen the user adds energy to the system; moving a finger (or multiple fingers) across the screen overrides the fluid dynamic system by forcing the vector to move in the indicated direction. Additionally, a user is able to choose saved presets, defined by him or her previously, to quickly jump to states and motions that they find compelling. The iPad application can also be used in performance situations, for example, by choreographers or VJs, to augment a dance performance or musical event, by connecting the iPad to a projector or monitor. Users can also augment the performance by superimposing video playback, a live video stream, or dynamic noise textures onto the fluid system.

Another deployment of Fluid Automata allows the user to view and interact with fluids in a virtual space via the iPad tablet. It utilizes the gyroscope sensor built-in to the iPad in order to transform the tablet's screen into a "magic lens" [9]. The fluid system is placed on the six sides of a cube map (also known as a "skybox" texture), a technique often used in video games to display landscapes or distant horizons. The user is imagined to be inside a virtual fluid system that they can see by spinning in place while rotating the tablet. By touching the screen, a ray is sent to the cube map texture, allowing the user to interact with the surrounding fluid system. Figure 13.8 shows a series of photos of a user interacting with the virtual fluid system.



Fig. 13.8 A user interacting with the virtual Fluid Automata installation using the iPad as a magic lens

13.5 Multi-user Interaction Techniques

In addition to the single-user interaction techniques described above, recent deployments of Fluid Automata enable multiple participants to interact with the system simultaneously. Research investigating interactive flow visualization indicates that engaging interaction techniques can successfully enable collaboration and encourage exploration [37]. In order to investigate these aspects of fluid simulation, a collaborative version of the Fluid Automata system was presented at the 2011 IEEE VisWeek Art Show [22]. In this installation, each user is given an iPad and invited to interact with a single fluid system projected large-scale on a wall. On each iPad, users see the current state of the fluid system's underlying vector representation (represented by rotating and scaling triangles) as well as an indication of the other user's interaction (represented by colored circles) [24].

The Fluid Automata system has also been ported to a multi-touch table, making it easier for multiple people to interact with the system at the same time, and to explore specific gestures that were defined to allow users to alter the intensity of the turbulence of the system. In this installation, touches on the table are mapped to inputs that update the fluid vectors. The table uses a perimeter of LED light sources and light detection sensors to detect touches on the surface of the table. Figure 13.9 shows an image of people interacting with the multi-touch table running an iteration of the Fluid Automata project.

The Fluid Automata system has also been used as an instrument for controlling a multimedia composition, *Study in Brownian (F*) Motion*, in collaboration with the composer Kiyomitsu Odai. The visual output of the composition is projected

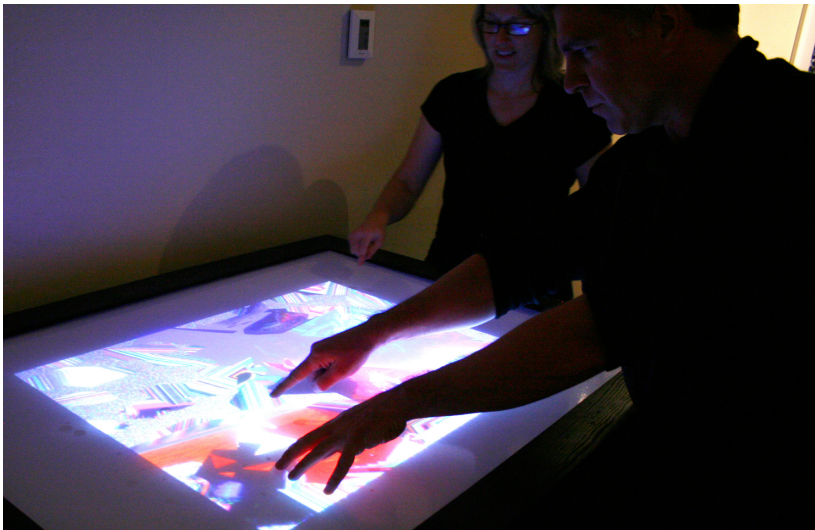


Fig. 13.9 Users gathered around a multi-touch table running a project that uses the Fluid Automata system

onto a large display and the audio output is channeled through multiple speakers. In addition to control via multi-touch, the system responds to Open Sound Control (OSC) [65] messages generated from musical events created on another computer and sent over the network. Additionally, fluid vectors and various fluid parameters can be transmitted wirelessly via OSC to influence the composition.

13.6 The *Annular Genealogy* Project

Annular Genealogy is more ambitious project based on generative systems that involves multi-user interaction. It is an interactive multimedia artwork for two performers using multi-channel speakers, a projector, and a tablet computer. The audio and visual components of the artwork explore feedback processes that the performers interactively shape into appealing, transient structures. The composition engine generates output via a stochastic sequencer that uses Brownian motion as a guiding metaphor. Similarly, the visualization engine, based on the Fluid Automata system, depicts colored fluid energy as a representation of dynamic, ephemeral structures. In addition to exploring these feedback processes independently of each other, each engine also directly influences the other via networked communication: both the visual and audio processes broadcast data via OSC messages which then influence various parameters of the composition and/or visualization. Finally, even the physical interactions are fed into the generative system as contact microphones are used to pick up the tapping and other ambient sounds made during the interaction. The performance aims to bring the layers of feedback into a cohesive compositional experience. These feedback layers are interconnected, and include: the generation of new musical motifs being created from the processing of the output sound; the generation of visual forms from the processing of the output graphics; the vector positions that govern the displacement of the visual forms used as inputs to control music parameters; and the sequencing parameters controlling the generation of the composition used as inputs to control image processing parameters [26].

Other recent multimedia installations have also featured generative compositions that make use of feedback mechanisms between the audio and visual components. For instance, Karen Curley's *Licht und Klang* is an audio-visual installation that generates sounds via optical sensors that use the refractions of light through oil and water as inputs into sound generation software [18]. A work by Joel Ryan and Ray Edgar called *LINA* features both musical and visual output based on mappings from a single CA system [14, 21]. Various electro-acoustic ensembles have explored the use of networked feedback as a tool for improvised performance. Most famously, the new media ensemble, The Hub, creates multimedia performances based on sets of rules that transform signals passed between performers and that are then presented in aural and visual domains [35]. *Annular Genealogy* similarly creates synaesthetic output based on a syncretic fusion of a mixed audio and visual feedback loop.

By supplying a multi-touch and live coding environment as an interface to and influencer of the generative processes another layer of feedback is added in which the performer responds to and shapes the multimedia output. That is, the performers

are *participants* in a compositional process. The generative software serves to create some structures independently of the performers; it is the role of the performers to guide the generational processes toward more compositionally interesting output and away from output that is overly repetitive, monochromatic, garish, or otherwise less satisfactory. Likewise, the audio and visual engines, via the various feedback processes, continually push against the explicit control of the performers. Overall, the composition is defined by a network of nested feedback loops that link the performer and the algorithm to create an inherent aesthetic tension between the generative and the interactive, the performed and the composed, the random and the intended. Figure 13.10 shows audience members inside an installation of *Annular Genealogy* in the AlloSphere Research Facility, an immersive CAVE-like environment housed at UC Santa Barbara [2].

While some of the results of interconnecting multiple feedback layers are unpredictable, the performers nonetheless begin to have an intuition as to how their actions will update the overall composition. For example, while there is no direct mapping of how the visualization data will update the compositional structures, after some experience using the iPad interface, it becomes clear that certain gestures during certain kinds of passages generate a particular shaping of the composition. It was also interesting to re-conceive the performers role as “guiders” of aesthetics, rather than as creators. A direction for future versions of the artwork would be to more explicitly highlight the effect that an interaction has as it is transmuted from one medium to the other.



Fig. 13.10 Photograph of viewers wearing 3D active stereo glasses within an installation of *Annular Genealogy* inside the AlloSphere Research Facility at UC Santa Barbara

13.7 Conclusion

This chapter discussed the integration of CA systems into creative projects and presented some of the design decisions that were made to facilitate effective interactivity in standalone and collaborative deployments. Although CA systems are conceptually simple, relatively minor modifications in rule sets can facilitate a wide range of variation. Many creative projects that utilize CA systems tend to either directly represent the CA system or offer a direct mapping of its inputs to an audio or visual output. This chapter instead explored two art projects that use CA as a creative mechanism that generates output which does *not* resemble the CA input. In particular, these two projects enable interaction and improvisation with CA and other generative systems. Fluid Automata extends CA into a more complex system in order to function as an interactive fluid simulation; and Annular Genealogy uses the Fluid Automata system as one input among other iterative feedback processes to generate a dynamic creative experience. By describing these two projects in some detail, we offer an example of how media arts projects may effectively extend CA concepts. The use of generative systems continues to intrigue artists, and new creative projects will incorporate CA system in novel and unexpected ways.

Appendix: Cell-State Transition Algorithm

The following code listing describes the fluid simulation, based on CA, as described in section 3.1. The *cell* parameter refers to a cell positioned in the CA grid (*pos*), with a particular state described by a vector (*vec*) that has a particular magnitude (*mag*) and orientation (*ang*). The *grid* has a pixel width of *width* and pixel height of *height*, and a particular number of columns (*cols*) and rows (*rows*), and contains a double array of cells. The variable *m* refers to the ratio that splits a cell vector into forward and “orthogonal” streams, the momentum parameter, or r_1 . The variable ϕ refers to the angularity parameter. Other optional variables, such as the directionality and viscosity parameters, have been excluded for clarity. Note that the NEIGHBORCELLS procedure retrieving a cell’s neighbors also includes the cell itself. Finally, the INTERSECTRECTS procedure (referred to but not shown) returns a value between 0.0 and 1.0 indicating the amount that two rectangles of the same size overlap.

Algorithm : UPDATECELLVECTOR($cell, grid, m, \phi$)

comment: Update vector vec in a Cell c by querying neighbors in Grid g

procedure NEIGHBORCELLS(c, g)

$neighborCells \leftarrow \text{ARRAY}(9)$

for $i \leftarrow -1$ **to** 1

for $j \leftarrow -1$ **to** 1

do $\begin{cases} nc \leftarrow c.col + i \\ nr \leftarrow c.row + j \\ \text{if } nc < 0, nc \leftarrow grid.cols - 1 \\ \text{if } nc > grid.cols - 1, nc \leftarrow 0 \\ \text{if } nr < 0, nr \leftarrow grid.rows - 1 \\ \text{if } nr > grid.rows - 1, nr \leftarrow 0 \\ neighborCells.push(grid[nc][nr]) \end{cases}$

return ($neighborCells$)

procedure MAKERECT(p)

$w \leftarrow g.width/g.cols$

$h \leftarrow g.height/g.rows$

$pLL \leftarrow vec2(p.x - w/2, p.y - h/2)$

$pUR \leftarrow vec2(p.x + w/2, p.y + h/2)$

return (RECT(pLL, pUR))

procedure PARTIAL(pos, nc, nv)

$p \leftarrow \text{INTERSECTRECTS}(\text{MAKERECT}(pos), \text{MAKERECT}(nc.pos + nv))$

return ($nv * p$)

procedure STREAMVECTOR(c, nc, mr, ao)

return ($vec2(nc.mag * mr, nc.ang + ao)$)

main

$mag \leftarrow ang \leftarrow 0.0$

$p \leftarrow cell.pos$

$arr \leftarrow \text{NEIGHBORCELLS}(cell, grid)$

for $i \leftarrow 0$ **to** 9

do $\begin{cases} nc \leftarrow arr[i] \\ fv \leftarrow \text{STREAMVECTOR}(cell, nc, m, 0.0) \\ lv \leftarrow \text{STREAMVECTOR}(cell, nc, 1.0 - m/2.0, -\phi) \\ rv \leftarrow \text{STREAMVECTOR}(cell, nc, 1.0 - m/2.0, \phi) \\ v \leftarrow \text{PARTIAL}(p, nc, fv) + \text{PARTIAL}(p, nc, lv) + \text{PARTIAL}(p, nc, rv) \\ mag \leftarrow mag + v.mag \\ ang \leftarrow ang + v.ang \end{cases}$

$c.vec \leftarrow vec2(mag, ang)$

References

1. Aceti, L., Milovac, T.: Geometries of the sublime. *Leonardo Electronic Almanac* ("ISEA2011 Istanbul: Uncontainable") 18(5), 329–344 (2011)
2. Amatriain, X., Kuchera-Morin, J., Hollerer, T., Pope, S.T.: The AlloSphere: Immersive multimedia for scientific discovery and artistic exploration. *IEEE MultiMedia*, 64–75 (2009)
3. Ashley, R.: Spore at the cellular level, <http://www.lup.com/features/spore-cellular-level> (accessed November 2013)
4. Aiken, M.: MSA fluid demos (2009), <http://www.memo.tv>
5. Babcock, J.: Cellular automata method for generating random cave-like levels (2005), <http://roguebasin.roguelikedevdevelopment.org>
6. Baggio, D.L., Emami, S., Escrivá, D.M., Ievgen, K., Mahmood, N., Saragih, J., Shilkrot, R.: *Mastering OpenCV with practical computer vision projects*. Packt (2012)
7. Beyls, P.: Musical morphologies from self-organizing systems. *Journal of New Music Research* 19(2-3), 205–218 (1990)
8. Beyls, P.: Cellular automata mapping procedures. In: *Proceedings of the International Computer Music Conference*, pp. 55–58 (2004)
9. Bier, E.A., Stone, M.C., Pier, K., Buxton, W., DeRose, T.D.: Toolglass and magic lenses: the see-through interface. In: *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques*, pp. 73–80. ACM (1993)
10. Boden, M.A., Edmonds, E.: What is generative art? *Digital Creativity* 20(1-2), 21–46 (2009)
11. Borsani, C., Cattaneo, G., Mattei, V., Jocher, U., Zampini, B.: 2d and 3d lattice gas techniques for fluid-dynamics simulations. In: Bandini, S., Serra, R., Liverani, F. (eds.) *Cellular Automata: Research Towards Industry*, pp. 67–79. Springer (1998)
12. Brown, P.: Emergent behaviours towards computational aesthetics. *Artlink* 16(2–3) (1996)
13. Burraston, D.: Generative music and cellular automata: An introduction to the online bibliography. *Leonardo* 45(2), 165–165 (2012)
14. Burraston, D., Edmonds, E.: Cellular automata in generative electronic music and sonic art: a historical and technical review. *Digital Creativity* 16(3), 165–185 (2005)
15. Burt, W.A.: Cellular automata as spectra: Beyond sonification into composition. In: Riddell, A., Thorogood, A. (eds.) *Proceedings of the Australasian Computer Music Conference*, pp. 27–33. Kyoto, Japan (2007)
16. Cappuccio, R., Cattaneo, G., Ciucci, D., Jocher, U.: CA fluid dynamics simulation paradigms lattice gas, digital fluid, lattice boltzmann: A comparison. In: Bandini, S., Worsch, T. (eds.) *Theory and Practical Issues on Cellular Automata*, pp. 20–28. Springer (2001)
17. Ceric, V.: Algorithmic art: Technology, mathematics and art. In: *30th International Conference on Information Technology Interfaces, ITI 2008*, pp. 75–82. IEEE (2008)
18. Curley, K.: *Licht und klang* (2009), <http://vimeo.com/5333718>
19. Dewdney, A.K.: Computer recreations. *Scientific American* 259, 104–107 (1988)
20. Dorin, A.: Liquiprism: Generating polyrhythms with cellular automata. In: *Proceedings of the International Conference on Auditory Display (ICAD)*, vol. 5, Kyoto, Japan (2002)
21. Edgar, R., Ryan, J.: Lina. In: *Exhibition of the 1986 International Computer Music Conference, ICMA* (1986)
22. Forbes, A.G.: Fluid Automata. In: Keefe, D., Campbell, B., Thorson, L. (eds.) *IEEE VisWeek 2011 Art Show Catalog* (2011)

23. Forbes, A.G.: Fluid automata, <http://fluidautomata.com> (accessed November, 2013)
24. Forbes, A.G., Höllerer, T., Legrady, G.: Expressive energy: The fluid automata project. In: Proceedings of the International Symposium on Electronic Art (ISEA), Albuquerque, New Mexico, pp. 65–70 (2012)
25. Forbes, A.G., Höllerer, T., Legrady, G.: Generative fluid profiles for interactive media arts projects. In: Proceedings of the International Symposium on Computational Aesthetics in Graphics, Visualization, and Imaging (CAe), Anaheim, California, pp. 123–129 (2013)
26. Forbes, A.G., Odai, K.: Iterative synaesthetic composing with multimedia signals. In: Proceedings of the International Computer Music Conference (ICMC), Ljubljana, Slovenia, pp. 573–578 (2012)
27. Galanter, P.: What is generative art? Complexity theory as a context for art theory. In: Proceedings of the Generative Art Conference, GA (2003)
28. Galanter, P.: Generative art and rules-based art. *Vague Terrain* 3 (2006)
29. Galanter, P.: Xepa. In: Forbes, A.G., Thorson, L. (eds.) *The IEEE VIS 2013 Art Show Catalog*, pp. 4–5 (2013)
30. Galanter, P.: Xepa: Intelligent sculptures as experimental platforms for computational aesthetic evaluation. In: Proceedings of the IEEE VIS Arts Program (VISAP), IEEE (2013)
31. Gánti, T.: Biogenesis itself. *Journal of Theoretical Biology* 187(4), 583–593 (1997)
32. Gánti, T.: *The Principles of Life*. Oxford University Press (2003)
33. Gardner, M.: Mathematical games: The fantastic combinations of John Conway's new solitaire game "life". *Scientific American* 223(4), 120–123 (1970)
34. Greene, D.: Re: Geminoid challenges (November 23, 2013), <http://www.conwaylife.com/forums/viewtopic.php?f=2&t=1006&p=9917#p9901>
35. Gresham-Lancaster, S.: The aesthetics and history of the hub: The effects of changing technology on network computer music. *Leonardo Music Journal*, 39–44 (1998)
36. Guay, M., Colin, F., Egli, R.: Simple and fast fluids. In: Engel, W. (ed.) *GPU Pro 2*, pp. 433–444 (2011)
37. Isenberg, T., Hinrichs, U., Carpendale, S.: Studying direct-touch interaction for 2d flow visualization. In: *Collaborative Visualization on Interactive Surfaces-CoVIS 2009*, p. 17 (2009)
38. Kiwiwonga: Another cellular automaton video (April 5, 2011), www.gamedev.net/blog/844/entry-2249737-another-cellular-automaton-video
39. Lafia, M.: Algorithms and allegories. *Digital Creativity* 14(2), 125–128 (2003)
40. Lala, R.: *Morphon* (2011), <http://www.riteshlala.net>
41. Legrady, G.: George legrady studio: Algorithmic visualizations, <http://www.georgelegrady.com> (accessed November 2013)
42. Millen, D.: Cellular automata music. In: Proceedings of the 1990 International Computer Music Conference, pp. 314–316 (1990)
43. Motte, W.F.: *Oulipo: a primer of potential literature*. University of Nebraska Press Lincoln, London (1986)
44. O'Rourke, K.: *Walking and Mapping: Artists as Cartographers*. MIT Press (2013)
45. Prusinkiewicz, P., Lindenmayer, A., Hanan, J.S., Fracchia, F.D., Fowler, D.R., de Boer, M.J., Mercer, L.: *The algorithmic beauty of plants*. Springer (1990)
46. Rafler, S.: Generalization of Conway's "game of life" to a continuous domain-smoothlife. arXiv preprint arXiv:1111.1567 (2011)
47. Reas, C.: *Compendium 2004-2010*. REAS Studio (2010)

48. Reas, C.: A database for Casey Reas, <http://reas.com> (accessed November 2013)
49. Roads, C.: The computer music tutorial. MIT Press (1996)
50. Roberts, C.: Control: Software for end-user interface programming and interactive performance. In: Proceedings of the International Computer Music Conference (ICMC), pp. 425–428. Huddersfield, UK (2011)
51. Roberts, C., Forbes, A., Höllerer, T.: Enabling multimodal mobile interfaces for interactive musical performance. In: Proceedings of the International Conference on New Interfaces for Musical Expression (NIME), pp. 102–105. Daejeon, Korea (2013)
52. Shanken, E.A.: Art in the information age: Technology and conceptual art. *Leonardo* 35(4), 433–438 (2002)
53. Shanken, E.A.: Cybernetics and art: Cultural convergence in the 1960s. In: Clarke, B., Henderson, L.D. (eds.) *From Energy to Information*, pp. 155–177. Stanford University Press, Palo Alto (2002)
54. Solomos, M.: Cellular automata in Xenakis's music. Theory and practice. In: *Definitive Proceedings of the International Symposium Iannis Xenakis*, Athens (May 2005–2006)
55. Stam, J.: Stable fluids. In: Proceedings of the 26th annual conference on Computer graphics and interactive techniques, pp. 121–128. ACM Press/Addison-Wesley Publishing Co (1999)
56. Stam, J.: Real-time fluid dynamics for games. In: Proceedings of the game developer conference, vol. 18 (2003)
57. Telea, A., van Wijk, J.: 3d ibfv: Hardware-accelerated 3d flow visualization. In: Proceedings of IEEE Visualization (VIS), pp. 233–240. IEEE Computer Society (2003)
58. Torrens, P.M., Benenson, I.: Geographic automata systems. *International Journal of Geographical Information Science* 19(4), 385 (2005)
59. Van Wijk, J.: Image based flow visualization. *ACM Transactions on Graphics (TOG)* 21(3), 745–754 (2002)
60. Vichniac, G.: Cellular-automata fluids. In: Tirapegui, E., Villarroel, D. (eds.) *Instabilities and Nonequilibrium Structures II, Mathematics and Its Applications*, vol. 50, pp. 97–116. Springer (1989)
61. Vorn, B.: *Evil / live* (1997), <http://billvorn.concordia.ca/robography>
62. Wakefield, G., Ji, H(H.): Artificial nature: Immersive world making. In: Giacobini, M., Brabazon, A., Cagnoni, S., Di Caro, G.A., Ekárt, A., Esparcia-Alcázar, A.I., Farooq, M., Fink, A., Machado, P. (eds.) *EvoWorkshops 2009. LNCS*, vol. 5484, pp. 597–602. Springer, Heidelberg (2009)
63. Wolfram, S.: Cellular automaton fluids 1: Basic theory. *Journal of Statistical Physics* 45(3–4), 471–526 (1986)
64. Wolfram, S.: *A New Kind of Science*. Wolfram Media, Inc., Champaign (2002)
65. Wright, M.: Open sound control: An enabling technology for musical networking. *Organised Sound* 10(3), 193–200 (2005)
66. Wuensche, A.: The ghost in the machine: Basins of attraction of random boolean networks. In: Langton, C. (ed.) *Artificial Life III, Santa Fe Institute Studies in the Sciences of Complexity*, Addison-Wesley (1994)
67. Xenakis, I.: *Formalized Music: Thought and Mathematics in Composition*. Pendragon Press (1992)